

PIVLab Tutorial

Madie Mamer, Summer 2026

1 Tips, Warnings, and Advice

During the experimental setup, make sure to note the frames per second setting on the camera. Either during the experimental recording or immediately after in a new video, hold up a ruler to get the pixel dimensional scaling. Do not do this with an image because the images and videos taken by a camera have different focal lengths and therefore different pixel scalings.

Sometimes the PIVlab GUI window disappears. With the MATLAB window open, navigate to “Window” in the top bar of your desktop and in the drop down menu select the PIVlab GUI.

PIVlab does not overwrite .txt files, the software instead appends to them. If you are redoing an analysis and re-exporting the data, delete the previous .txt files or move them to a "backup" folder.

To install Python dependencies via terminal use: `python3 -m pip install packageName`

2 Decomposing video footage into frames

Before beginning the pre-processing with the PIV software, the video footage must be decomposed into frames. To do so, navigate to the directory containing the video footage on your terminal and then execute the "extract_frames.sh" script via:

```
extract_frames.sh videoName.mp4
```

replacing "videoName.mp4" with the name of your video file. This automatically creates a directory called "frames" and names each frame as "frame_%05d".

3 Processing with PIVLab

PIVLab is a software built on MATLAB that has a user-friendly GUI for quick processing of PIV images. Here are the steps to download the software:

1. Download MATLAB using the university licensing.
2. During the download process, select packages “Parallel Computing Toolbox” and “Image Processing Toolbox”.

3. Once downloaded, open MATLAB and navigate to the “Apps” button on the top bar. Then navigate to “Get more Apps”. A second window will appear. Using the search bar in this window, search “PIVLab”. Install it.
4. To launch PIVLab, navigate to the “Apps” button the top bar. There should be an icon with “PIVLab_GUI” underneath it. Click it. This will open the GUI as a secondary window.

During step 4, there may be additional packages that need to be installed depending on your MATLAB version. Install them directly via the command line interface. When the PIVLab GUI opens, you will be asked:

"PIVlab can be run with parallel computing.

- Recommended when processing multiple images.
- Not required when acquiring images or processing mp4 and avi files.

Open parallel pool?"

Click yes. It may take a few moments for “Changing parallel pool” to complete.

3.1 Importing Images

On the left panel, click “Import Images.” Navigate to the directory containing the images you decomposed in the first section of the tutorial. For this tutorial, we will work with the Jet example provided by the PIVLab Package. In the import images window, navigate to the directory containing the tutorials. For mac users that is likely to be in:

"/Users/USERNAME/Library/Application Support/MathWorks/MATLAB Add-Ons/Toolboxes/PIVlab/Example_data/".

Scroll to the bottom and highlight all images with the name “Jet”, there should be twenty total. For the tutorial, we will leave the “Image Sequencing Style” radio button as “Pairwise”, however, for image data from our laboratory we will use “Time resolved”. “Pairwise” sequencing utilizes two cameras that take successive photos with short time durations in between. “Time resolved” is used for photos from a single camera in a continuous video fashion. Confirm you have the right “Image Sequencing Style” setting. Click “Import”.

3.2 Pre-Processing the Images

Navigate to “Image settings” in the top bar of the PIVLab GUI. Click “Image pre-processing/enhancement”. Make sure “Enable CLAHE” is checked and set the window size to 64. Mess around with “Enable Highpass”, “Auto Contrast Stretch”, and “Background subtraction” until you the image has qualitatively good contrast and clarity. You may not always need to use these options.

Navigate to “Image settings” in the top bar of the PIVLab GUI. Click “Define region of interest (ROI)”. Either set your coordinates directly or click “Select ROI” in the left panel and draw a bounding box directly on the image. Record the x, y, width, and height of the bounding box to ensure the ROI is kept consistent between frames.

Navigate to “Image settings” in the top bar of the PIVLab GUI. Click “Define masks (exclude regions from analysis)”. This step will be crucial for removing the ice from the analysis, which can provide erroneous velocity values and consume computational time and memory for the PIV analysis. Use the “Free hand” tool within the “Polygon mask items” section of the left panel to draw the mask on the image. In this tutorial, we will mask out the nozzle on the far right of the image. It is only necessary to mask out the region within your ROI. Draw the mask by clicking and dragging. You may go back and edit individual vertices or add vertices by double clicking on the path line. Once you are satisfied with the masked region, click “Copy mask to frames” under the “Mask alterations” section in the left panel.

3.3 Analyzing the Images

Next, we will navigate to the “Analysis” tab in the top bar. From the drop down, select “PIV settings”. Ensure that the “PIV algorithm” is set to “Multipass FFT window deformation”. It is recommended to use all four passes. Each “pass” is an independent run of the algorithm that evaluates the correlation matrix. Pass 1 should have the largest “Interrogation area” working down to smaller areas through Passes 2, 3, and 4. Start by setting Pass 4 to have an “Interrogation area [px]” of 32 pixels. Then set Pass 3 to 64 pixels and Pass 2 to 128 pixels. Lastly, set Pass 1 set to 256 pixels. Set the “Step [px]” to be the difference between Pass 1 Interrogation area and Pass 2 Interrogation area, 128 pixels.

Click “Analyze current frame” to see how well the selected choices perform. Then, navigate to the “Validation” tab in the top bar. In the drop down menu select “Image based validation”. In the left panel, set the “Correlation coefficient filter Threshold” to be 0.1. Ensure this box is checked and “Interpolate missing data” is checked. Click “Apply to current frame”. A “Valid detection probability (VDP)” value will show up highlighted in bright green. This value should be 95% or higher. If it is below 85% the data is likely unusable and needs to be amended further.

Prior to analyzing all frames, alternate between validation and the analysis for a few frames spread throughout the to confirm the vitality of the entire experimental run. If most frames have a high VDP, then navigate to the “Analysis” tab in the top bar, and select “ANALYZE!” from the drop down. In the left panel, click “Analyze all frames”. Note that this step will likely take ~ 40 minutes for 5000 frames on a Mac Studio desktop. Once the analysis is completed, a Super Mario Bros-esque ring will sound. Navigate back to “Validation” and select “Image based validation” once more. Perform the validation check again and ensure majority of frames fall within the correct VDP. An alternative way to perform this check would be to select “Velocity based validation” from the “Validation” drop down and view the velocity scatter plot. In the left panel under “Velocity limits” select “Rectangle”. A new window with a scatter plot showing u vs v velocity will appear. Visually inspect if there are any crazy outliers.

3.4 Dimensional Calibration

Currently, all the distance values are in “pixel” world. If you click on one of the green vectors calculated during the Analysis section the velocity will show as a “px/fr” or pixel/frame value. To convert to “dimensional” world, we need to calibrate the image. Navigate to the “Spatial calibration (px $\rightarrow$ mm)” tab in the top bar. For the example, select “pick a reference length [px]” in the left panel. On the image, draw a line from the top of the nozzle to the bottom. This will auto populate the “Reference length in px” to be ~ 406 px. Set the “Real distance in mm” to be 50. Set the

“time step in ms” to $1000 \times 1/fps$. The typical frames per second (*fps*) used by the camera is 120, however, make sure to note this value when the experiment is being ran. the 1000 multiplier is to convert seconds to milliseconds. Under the “Setup Offsets” section in the left panel, ensure x increases to the right and y increases to the top. This makes the origin point (0,0) at the bottom left of the ROI. Click “Apply calibration”. Now click on a green vector on the image. The values shown should be “px/fr” and “m/s”.

3.5 Exporting and Saving

To export the data, navigate to the “File” tab on the top bar. In the menu drop down, select “Export” -> “Text file (ASCII)”. In the left panel, ensure the “Delimiter” is set to “comma”. Uncheck “Add file information”. “Add column headers” can be helpful, but not necessary. For the first few analyses, leave it checked until you become familiar with which column is which variable. Click “Export all frames”. Navigate to your analysis folder and create a new folder titled “textFiles” or something similar. Click “save”. Note that PIVLab fails to properly overwrite files. So, if you are redoing an analysis make sure to delete the previously written .txt files before exporting new ones.

Other tabs on the top bar include “Plot”, “Extractions”, “Statistics”, “Synthetic particle image generation”, and “Learn!”. For our purposes, we can ignore all of these. The “Plot” menu can be helpful for quick visualizations, however, it is recommended to switch over to your preferred plotting language for analysis and figure making.

Before closing PIVLab and MATLAB, navigate once more to “File” at the top bar. Select “Save” -> “Save PIVLab session”. This will save the choices made in the PIV analysis if you need to refer back to them or update them in the future. At this point, we are done using PIVLab and can navigate to a Python IDE for post-processing.

3.6 Plotting Transects

4 Starting from a previous session

It is recommended to “save your session” at the end of every analysis in case you need to revisit the parameters you set or adjust them. To load from a previous session open the PIVLab GUI as shown previously. Instead of “Import images” select “Load session”. Navigate to the directory where you stored your session data. It will be a '.mat' file. Select the file and click “Open”. If the PIVLab GUI window disappears, navigate to your computer’s top bar and click “Window”. The GUI window will be listed there underneath the main MATLAB window.

5 Python Post-Processing

5.1 Instantaneous Plotting

In the same directory as the analysis textFiles, create a new Jupyter Notebook titled “analysis.ipynb”. Import os, pandas, matplotlib, numpy, scipy.interpolate.griddata, and cmocool.cm (for pretty plotting). To open the analyzed .txt files use:

```
i = 1
ii = f'{i:04}'
```

```
df = pd.read_csv("textFiles/PIVlab_" + ii + ".txt", sep=",")
```

“PIVLab_0000” is the default PIVLab name and can be helpful to maintain in order to automate your plotting scripts.

Once the data is loaded, it needs to be converted to a 2D reference frame, which we will use numpy’s meshgrid and scipy’s griddata for. First, extract the variables:

```
x, y = df.iloc[:,0].values, df.iloc[:,1].values
u = df.iloc[:,2].values
v = df.iloc[:,3].values
speed = (u**2 + v**2)**(1/2)
```

Then, using np.meshgrid, construct a 2D mesh out of the 1D x and y arrays:

```
X, Y = np.meshgrid(x, y)
```

Now, using griddata regrid the 1D u, v, and speed data to the 2D mesh:

```
U = griddata((x,y), u, (X,Y))
V = griddata((x,y), v, (X,Y))
SPEED = griddata((x,y), speed, (X,Y))
```

Next the fun part, plotting! Here is an example of creating a three paneled figure showing u, v, and speed. For the velocity components u and v, use a ‘diverging’ colormap since values range across negative and positive values indicating different directions of motion. Make sure this colormap is centered at zero using matplotlib’s ‘mcolors.TwoSlopeNorm’. Centering at zero ensures that one color from the diverging colormap represents negative values and the other represents positive values. For plotting speed, use a ‘sequential’ colormap, such as plasma, viridis, jet, or cmocean’s thermal. Since speed is an absolute value, with only positive numbers, a sequential map is the most intuitive visualization.

```
fig, ax = plt.subplots(1,3,figsize=(20,5))

norm = mcolors.TwoSlopeNorm(vcenter=0, vmin=np.nanmin(U), vmax=np.nanmax(U))
c = ax[0].pcolormesh(x, y, U, cmap = cm.balance, norm = norm)
plt.colorbar(c, label = "U [m/s]")
ax[0].set_xlabel("X [m]"); ax[0].set_ylabel("Y [m]");

norm = mcolors.TwoSlopeNorm(vcenter=0, vmin=np.nanmin(V), vmax=np.nanmax(V))
c = ax[1].pcolormesh(x, y, V, cmap = cm.balance, norm = norm)
plt.colorbar(c, label = "V [m/s]")
ax[1].set_xlabel("X [m]"); ax[1].set_ylabel("Y [m]");

c = ax[2].pcolormesh(x, y, SPEED, cmap = 'jet')
plt.colorbar(c, label = "$\sqrt{u^2 + v^2}$ [m/s]")
ax[2].set_xlabel("X [m]"); ax[2].set_ylabel("Y [m]");

plt.tight_layout()
plt.savefig("Figures/velocity" + ii + ".png", dpi = 300)
```

5.2 Plotting the mean

In order to plot the mean, you have to combine all the text files in a smart manner. Luckily, we can leverage the fact that they all use the same grid. Below is an example loop that opens each file, extracts u and v , and iteratively sums the values together. Afterwards, the summed arrays are divided by the total file count, “ $nFiles$ ”.

```
files = [item for item in os.listdir('textFiles') if "PIVlab" in item]
nFiles = len(files)

for i in range(1,nFiles + 1):

    ii = f'{i:04}'
    print(f"Opening file index {ii}")
    # Open the text file.
    df = pd.read_csv("textFiles/PIVlab_" + ii + ".txt", sep = ",")

    if i == 1:
        x, y = df.iloc[:,0].values, df.iloc[:,1].values
        X, Y = np.meshgrid(x, y)
        u_avg = np.array(df.iloc[:,2].values)
        v_avg = np.array(df.iloc[:,3].values)
    else:
        u_avg += np.array(df.iloc[:,2].values)
        v_avg += np.array(df.iloc[:,3].values)

u_avg = u_avg/nFiles
v_avg = v_avg/nFiles
speed_avg = (u_avg**2 + v_avg**2)**(1/2)

# Grid variable data to 2D meshgrid.
U = griddata((x,y), u_avg, (X,Y))
V = griddata((x,y), v_avg, (X,Y))
SPEED = griddata((x,y), speed_avg, (X,Y))

fig, ax = plt.subplots(1,3,figsize=(20,5))

norm = mcolors.TwoSlopeNorm(vcenter=0, vmin=np.nanmin(U), vmax=np.nanmax(U))
c = ax[0].pcolormesh(x, y, U, cmap = cm.balance, norm = norm)
plt.colorbar(c, label = "U [m/s]")
ax[0].set_xlabel("X [m]"); ax[0].set_ylabel("Y [m]");

norm = mcolors.TwoSlopeNorm(vcenter=0, vmin=np.nanmin(V), vmax=np.nanmax(V))
c = ax[1].pcolormesh(x, y, V, cmap = cm.balance, norm = norm)
plt.colorbar(c, label = "V [m/s]")
ax[1].set_xlabel("X [m]"); ax[1].set_ylabel("Y [m]");
```

```

c = ax[2].pcolormesh(x, y, SPEED, cmap = 'jet')
plt.colorbar(c, label = "$\sqrt{u^2 + v^2}$ [m/s]")
ax[2].set_xlabel("X [m]"); ax[2].set_ylabel("Y [m]");

plt.tight_layout()
plt.savefig("Figures/velocity_average.png", dpi = 300)

```

5.3 Plotting Transects

For most of our analyses, it will be helpful to plot transects of key variables along the ice-normal direction. Below is a script that demonstrates how to do this, extracting three transects along the direction of the jet.

```

nY, nX = U_avg.shape
n_slices = 3
# y = np.sort(y)
# x = np.sort(x)
fig, ax = plt.subplots(1, 3, figsize=(20, 5))

# --- Row 0: horizontal slices (fixed y, profile vs x) ---
inc_y = max(1, nY // n_slices)
y_idx = list(range(0, nY, inc_y))
cmap0 = cm.phase(np.linspace(0, 1, len(y_idx) + 1))
for c, i in zip(cmap0, y_idx):
    ax[0].plot(x, U_avg[i, :], color=c, label=f"y={y[i]:.3f}")
    ax[1].plot(x, V_avg[i, :], color=c, label=f"y={y[i]:.3f}")
    ax[2].plot(x, SPEED_avg[i, :], color=c, label=f"y={y[i]:.3f}")

ax[0].set_title("U(x)");    ax[1].set_title("V(x)");    ax[2].set_title("Speed(x)")

for a in ax[:]:
    a.set_xlabel("X [m]")

ax[0].set_ylabel("U [m/s]")

ax[1].set_ylabel("V [m/s]")

ax[2].set_ylabel("Speed [m/s]")

for a in ax.flat:
    a.grid(alpha=0.1)
    a.legend(fontsize=7, ncol=2)

plt.tight_layout()
plt.savefig("Figures/velocity_transect.png", dpi = 300)

```